

Teaching a Computer Connect Four

Learning notes for steps 1 to 3 of the self-play agent project: the game, tree search, and the neural network

These notes explain what was built in the first three steps of the **connect4-alphazero** project, the reinforcement-learning ideas behind each step, and what the results showed. Each idea comes with an everyday example.

The goal

An agent that learns to play Connect Four well using nothing but games against itself. This is the method behind AlphaZero, the system that mastered chess and Go. Nobody shows it good moves; it has to discover them.

The seven steps

Step	What it adds	Status
1. The game	The rules: the world the agent lives in	Done
2. Search without learning	Planning by imagining thousands of random games	Done
3. The neural network	Instinct: judging a board in one glance	Done
4. Search guided by the network	Instinct and planning working together	Next
5. Self-play training	The agent plays itself and learns from the games	To do
6. Measuring strength	Ratings across training, several runs	To do
7. A playable demo	Play against the trained agent	To do

How the three finished steps fit together

Step 1 gives us a world with rules. Step 2 gives us a player that is strong but slow and forgetful, because it works everything out from scratch each move. Step 3 gives us a player that is fast and remembers, but is weaker because it cannot look ahead. Step 4 will join the two, and that combination is what makes this method powerful.

Step 1: The game

The idea: every RL problem starts with an environment

In reinforcement learning, the **environment** is the world the agent acts in. It defines three things, and everything the agent ever learns comes from them.

Term	Meaning	In Connect Four
State	Everything needed to decide what to do next	The board, plus whose turn it is
Action	A choice the agent can make	Which of the 7 columns to drop a disc into
Reward	The feedback the agent gets	Only at the end: +1 for a win, -1 for a loss, 0 for a draw

Real-world example. Think of a tournament chess game. The position on the board is the state, a legal move is an action, and the result slip handed in at the end is the reward. Nobody tells you during the game whether move 12 was good. You only find out when the game is over.

Why this is harder than it looks: credit assignment

The agent gets one number after 20 to 40 moves and has to work out which of those moves deserved the credit or the blame. This is called the **credit assignment problem**. In the earlier trading project the agent received a reward every single day, so this problem barely arose. Here it is the central difficulty, and it is the reason the project needs search (step 2) and a value estimate (step 3).

Real-world example. A football team loses 1-0. Was it the defender's slip in the 80th minute, the missed chance in the 10th, or the manager's line-up? The scoreboard gives one number for ninety minutes of decisions.

What was built (src/azc4/game.py)

The board is a grid of 6 rows and 7 columns holding numbers: +1 for the first player's discs, -1 for the second player's, 0 for empty.

Playing a move returns a new position and leaves the old one untouched. In step 2 the agent imagines thousands of possible futures from the same position. If playing a move changed the board in place, those imagined futures would corrupt each other.

Real-world example. A chess player thinking 'if I go here, they go there' does not move the real pieces. Every line they imagine starts again from the same real board.

Win detection only checks lines through the disc just placed. A new four-in-a-row must include the new disc, so there is no need to scan the whole board. This keeps the game fast (about 27,000 moves per second), which matters because training needs millions of moves.

The **'canonical' view shows the board from the side of whoever is about to move.** Their own discs always appear as +1 and the opponent's as -1. This lets a single network play both sides: it only ever answers 'what should I do here?'.

Real-world example. A football coach draws tactics as 'us' and 'them', not as 'the team in blue'. The same advice works whichever kit you happen to be wearing.

Testing the rules

An agent trained on a buggy game learns the bugs, so the rules get the most testing: wins in all four directions for both players, full columns rejecting moves, a known 42-move game that ends in a draw, and 300 random games compared against a slow but obviously correct win checker that examines every possible line.

First finding: going first is an advantage

In 2,000 games between two players moving completely at random:

Outcome	Share of games
First player wins	56.8%
Second player wins	43.1%
Draw	0.1%
Average game length	21 moves

Even with no skill at all, the first player wins more often. So whenever two players are compared later, each must go first in exactly half the games. Otherwise the match would be measuring the seat, not the player.

Step 2: Search without learning

The idea: plan by imagining games

Before each move, the player imagines thousands of complete games starting from the current position and keeps track of which first move led to the most wins. Then it plays that move. This is called **Monte Carlo tree search**, or MCTS. Nothing is learned and nothing is remembered between moves.

Real-world example. You are choosing between three routes to work and cannot calculate which is fastest. So you drive each one many times and note how long it took. You would not split your trips evenly for ever: once route B looks best you mostly drive B, but you still try A and C now and then in case your first impressions were unlucky. MCTS does exactly this, with moves in place of routes.

The four phases of one imagined game

Phase	What happens
1. Select	Starting from the current position, walk down the moves already explored, each time picking the one that best balances 'has won often' against 'has hardly been tried'.
2. Expand	On reaching a position with a move not yet tried, add that move to the tree.
3. Simulate	From there, play random moves until the game ends.
4. Back up	Pass the result back up the path, so every position on it records one more game and who won.

After all the simulations, the player chooses the move that was **tried most often**. Good moves attract more simulations, so the visit count is the search's verdict. This is more reliable than picking the highest win rate, which could belong to a move tried twice that got lucky.

The key idea: exploration versus exploitation

- **Exploit:** keep trying the move that has worked best so far.
- **Explore:** try other moves in case one is better than it first looked.

Every learning agent faces this trade-off. The trading agent handled it crudely, by taking a random action some fixed share of the time (epsilon-greedy). MCTS uses a smarter rule called **UCB** (upper confidence bound). Each move's score is its average result so far plus a bonus. The bonus is large when the move has rarely been tried and shrinks as it gets tried more.

Real-world example. A new restaurant with two reviews averaging 5 stars is not clearly better than one with 500 reviews averaging 4.6. UCB gives the two-review place the benefit of the doubt, so it gets another visit. If it disappoints, the bonus fades and you go back to the reliable one.

Watching it think

In this position X has three in a row along the bottom and it is O's turn. Any move except column 3 loses immediately.

```
0 0 . . . . .
X X X . . . .
0 1 2 3 4 5 6
```

After 2,000 simulations the search had spread its effort like this:

Column	Times tried	Estimated chance that O wins
0	17	18%
1	24	25%
2	24	25%
3	1,873	61%
4	20	20%
5	22	23%
6	20	20%

It tried each losing move about 20 times, saw them fail, and put 94% of its effort into the only move that survives. Nobody programmed it to block. It found that out from the simulations.

How strong is it?

Scores count a draw as half a win. The interval shows how uncertain the score is given the number of games played.

Match	Result	Score (95% interval)
10 simulations vs random	133 wins, 67 losses	66.5% (59.7% to 72.7%)
50 simulations vs random	197 wins, 3 losses	98.5% (95.7% to 99.5%)
200 simulations vs random	100 wins, 0 losses	100% (96.3% to 100%)
200 vs 50 simulations	89 wins, 1 draw, 10 losses	89.5% (82.0% to 94.1%)
1,000 vs 200 simulations	48 wins, 1 draw, 11 losses	80.8% (69.1% to 88.8%)

- **More thinking means stronger play**, and it keeps helping well past the point of beating a random player.
- **The 1,000-simulation player is the baseline** that the learned agent must beat later. Having an honest baseline before building the clever thing is the same discipline used in the trading project.

What else was built

- **The arena (`src/azc4/arena.py`):** runs matches between any two players. Every player is simply 'a function that takes a position and returns a column', so a random mover, a search and a neural network can all be compared in the same way. Players alternate who goes first.
- **Confidence intervals on win rates:** 8 wins out of 10 could easily be luck; 80 out of 100 is much stronger evidence. The interval says how much to trust a score.

The weakness that leads to step 3

The simulations use random moves, and random play is a poor guide to what strong players would do. So the search needs thousands of simulations to see through the noise. It also has no memory: on every move it starts from zero, even in a position it has met a hundred times before.

Real-world example. This player is like someone who has never played before but may try every idea on a practice board before each move. A grandmaster does not need to. They glance at the board and already know which two or three moves are worth considering, and roughly who is winning.

Step 3: The neural network

The idea: instinct in place of calculation

The search works everything out from scratch by playing thousands of random games. A neural network does the opposite: it looks at the board once and answers at once, from patterns it has learned.

Real-world example. A learner driver consciously works through every junction: check the mirror, check the speed, who has priority. An experienced driver simply sees the junction and knows. Nothing magical happened; thousands of past junctions were compressed into instinct. The network is that compression.

The network gives two answers

Output	The question it answers	Its form
Policy	Which moves look promising here?	A probability for each of the 7 columns
Value	Who is winning?	One number from -1 (the player to move loses) to +1 (wins)

Real-world example. A chess coach glancing at a student's game does two things at once: 'you should be looking at these two moves' (policy) and 'you are slightly worse here' (value). Both come from the same understanding of the position.

That is why this is **one network with two heads** and not two separate networks. A shared body learns to understand the board (where the threats are, who controls the centre), and two small heads turn that understanding into the two answers.

What was built (`src/azc4/network.py`)

Input encoding. The board is turned into two grids of 0s and 1s: 'where are my discs' and 'where are theirs'. With separate grids the network can look for 'my three in a row' and 'their three in a row' directly.

Convolutions. The body of the network slides a small 3 by 3 window across the board looking for patterns. A three-in-a-row is the same pattern in the bottom-left corner as in the top-right, so a convolution learns it once and recognises it everywhere.

Real-world example. Once you can recognise the letter 'A', you can read it anywhere on a page. You do not have to learn it again for each position.

Residual blocks. Each block adds its input back on to its output. This lets a layer learn 'the previous understanding, plus a small refinement', which is what makes deeper networks trainable. This network has 4 blocks and about 300,000 weights.

Real-world example. An editor improving a draft does not rewrite it from a blank page. They keep the draft and make corrections on top of it.

The loss function measures how wrong the network is, as two parts added together: how different its move preferences are from the target preferences (policy part), and how far its 'who is winning' was from what actually happened (value part). Training means adjusting the weights, little by little, to make this number smaller.

Mirror images. Connect Four is left-right symmetric, so flipping any position gives a second valid training example for free. This is called **data augmentation**.

The experiment: can the network learn by copying the search?

To check that the network can learn at all, it was trained to imitate the search from step 2.

- **Generate data.** The search (400 simulations per move) played 600 games against itself. For each of the 16,636 positions we saved how the search split its simulations across the columns, and who eventually won.
- **Train.** The network learned to give the same answers from one look.
- **Test.** The network then played with no search at all, choosing its favourite move each turn.

Real-world example. This is an apprentice watching a master at work and learning to copy the decisions, without yet understanding the reasoning behind them.

This is ordinary **supervised learning** (copying a teacher), not reinforcement learning yet. In step 5 the teacher becomes the network itself, and that is where the reinforcement learning begins.

A careful detail: the network was checked on 60 whole games it never saw, not on randomly chosen positions. Positions from the same game are near-copies of each other, so mixing them between training and checking would leak the answers. This is the same kind of leak the trading project guarded against.

Results: what the network learned

Measured on 60 unseen games	Before training	After training
Its favourite move matches the search's favourite	10.9%	49.4%
It names the eventual winner correctly	48.8%	61.1%

With 7 columns, picking at random would match about 14% of the time, so 49% is a large gain. Naming the winner at 61% is only modestly better than a coin flip.

Results: the network playing with no search

Opponent	Result over 100 games	Score (95% interval)
Random player	100 wins, 0 losses	100% (96.3% to 100%)
Search with 50 simulations	82 wins, 18 losses	82.0% (73.3% to 88.3%)
Search with 400 simulations (its teacher)	27 wins, 4 draws, 69 losses	29.0% (21.0% to 38.5%)

What these results mean

- **Instinct alone is already strong.** With a single glance and no looking ahead, the network never lost to a random player and clearly beat a search that imagines 50 games per move.
- **It is weaker than its teacher,** which is expected. It copies a search without being able to search.
- **The policy head learned well; the value head did not.** On unseen games the value error stayed above the level you would get by always answering 'even game', and it drifted upward during training, a sign of memorising. The reason is that 'who won this one game' is a very noisy label: the same position can lead to a win in one game and a loss in another.
- **This is exactly what the next steps address.** Self-play produces far more games, and the search gives the value head better targets.

Real-world example. The apprentice has picked up the master's habits ('in this kind of position, play in the centre') much faster than the master's judgement ('this position is winning'). Habits can be copied from watching. Judgement takes many more games.

What comes next (step 4)

Step 4 joins the two halves. The search will stop using random games and use the network instead: the policy tells it which moves are worth exploring, and the value tells it who is winning without having to play the game out. Instinct makes the planning efficient, and planning corrects the mistakes of instinct.

Glossary

Term	Plain meaning
Agent	The learner or decision-maker; here, the Connect Four player.
Environment	The world the agent acts in; here, the game and its rules.
State	Everything needed to choose the next action: the board and whose turn it is.
Action	A choice available to the agent: a column to play.
Reward	The feedback signal: +1, 0 or -1 at the end of the game.
Credit assignment	Working out which earlier decisions caused the final result.
Canonical view	The board shown from the side of the player about to move.
MCTS	Monte Carlo tree search: choosing a move by simulating many games.
Simulation (playout)	One imagined game played to the end.
Exploration	Trying options you are unsure about, to learn more.
Exploitation	Choosing the option that currently looks best.
UCB	A rule that scores each option as its average result plus a bonus for being rarely tried.
Policy	A rule for choosing actions; here, a probability for each column.
Value	An estimate of the final outcome from a position.
Convolution	A small pattern-detector slid across the whole board.
Residual block	A group of layers that adds its input back on to its output.
Loss	A number measuring how wrong the network is; training reduces it.
Supervised learning	Learning by copying correct answers supplied by a teacher.
Data augmentation	Creating extra training examples from existing ones, such as mirror images.
Held-out data	Examples kept aside from training, used to check real understanding.
Baseline	A simple reference player that a new method must beat to be worth having.

Try it yourself

From the project folder (C:\Users\ihamz\connect4-alphazero):

```
.venv\Scripts\python -m pytest
.venv\Scripts\python scripts\play.py --sims 1000
.venv\Scripts\python scripts\arena.py mcts:200 random
.venv\Scripts\python scripts\arena.py net:runs/imitate/net.pt mcts:50
.venv\Scripts\python scripts\imitate.py
```

The first runs the 28 tests. The second lets you play against the search and shows its thinking before each move. The third and fourth run matches between players. The last repeats the step 3

experiment.